

Philisa Abafazi Bethu

Volunteer App

Task 2 — Presentation Script & Running Order

Module	INSY7315 — Information Systems 3E, Work Integrated Learning
Client	Philisa Abafazi Bethu Women Centre SA, Lavender Hill, Cape Town
Team	Kwethu, Tlhogi, Ash
Runtime	11 minutes
Platform	Native Android, Kotlin, Firebase

How to use this. The italic lines are what to say. They are written to be spoken, not read out word for word — learn the shape of each one and use your own words on the day. The dark boxes say what must be on screen while you speak. Every section names the rubric row it is aimed at, so if you run short on time you know what you are giving up.

Before you record

Do all of this first. Half of these take a minute and losing marks to any of them would be painful.

- **Sign in on the emulator** as a volunteer, and have a second device or account signed in as admin. Switching accounts on camera wastes 40 seconds.
- **Seed the data.** Run the seed script so Activities and Community are full. Empty lists look like an unfinished app.
- **Create the Resizable emulator** (Device Manager → Resizable Experimental). It has a Display Mode dropdown for Phone / Unfolded / Tablet, so you can switch live on camera.
- **Open your tabs in advance:** the GitHub repo, the Actions tab with a green run, the Pull Requests tab, the Firebase console on the Rules page, and Android Studio.
- **Turn off notifications** on the recording machine. A Slack popup mid-demo reads as unprofessional.
- **Check the mic.** Poor audio is the single most common reason presentations lose marks on the Communication row.
- **Do one dry run** with the timer visible. Aim to finish 30 seconds early.

Two gaps to close before filming.

1. Ash has no commits. The repository history shows two contributors: Tlhogi (24 commits) and Kwethu (22). The GitHub & Version Control row is worth 10 marks and a marker who opens the contributors graph will see two names for a team of three. Get Ash's work committed under Ash's own account before submission, even if it is documentation, the ERD, or the test plan. Do not have someone else commit it for them.

2. There is no hosting yet. The Technical Demo row's top band asks for a "client-ready hosted solution". Right now there is no Firebase Hosting set up for this project. Put the signed APK on Firebase Hosting with a download page before you record, and show the live URL in Section 7. Without it the demo is capped below the top band.

Running order

Time	Who	Section	Rubric row it targets
0:00	Ash	Opening — the client and the problem	Communication (10)
0:45	Ash	Requirements alignment	Requirements Alignment (5)
1:30	Tlhogi	Architecture & system design	Architecture & System Design (15)
3:15	Kwethu	Demo 1 — the volunteer journey	Technical Demo (20)
5:15	Tlhogi	Demo 2 — admin, and the image API	Technical Demo (20), APIs (20)
6:45	Ash	Responsiveness & accessibility	Responsiveness (15)
7:45	Kwethu	Security, proven not claimed	Security (20), Architecture (15)
9:00	Kwethu	GitHub, testing and the pipeline	GitHub (10), CI/CD (20)
10:30	All	Hosting and close	Technical Demo (20)

The demo sections are the longest on purpose. Technical Demo & Implementation is the heaviest single row on the presentation sheet at 20 marks, and a smooth demo is also the evidence a marker trusts most.

1. Opening — Ash

0:00 – 0:45

Targets: Presentation & Communication (10 marks)

On screen: The app's welcome page, still. Do not move the mouse while talking.

"Good day. We are Kwethu, Tlhogi and Ash, and we built a volunteer management app for Philisa Abafazi Bethu Women Centre SA."

"Philisa is a non-profit in Lavender Hill, Cape Town. They have run since 2008 and they support around a hundred thousand people across Lavender Hill and five surrounding informal settlements, through ten programmes."

"Their problem was simple to describe and hard to live with. Volunteers were being coordinated by phone and by WhatsApp. Nobody had a single view of who was coming to what, and the team had no reliable way to reach their volunteers at once."

"What we will show you is a native Android app, built in Kotlin on Firebase, that is running now. Ash will cover the requirements, Tlhogi the architecture, and then we will demonstrate it."

Say the client's name properly and say what they do before anything technical. The Communication row rewards a presentation that sounds like it was made *for someone*.

2. Requirements alignment — Ash

0:45 – 1:30

Targets: Requirements Alignment (5 marks)

On screen: A single slide with two columns. Left: what the client asked for. Right: the feature that answers it.

"We took four requirements from the client and mapped each one to something you can open in the app."

"They needed volunteers to sign themselves up without staff doing admin. So there is no approval queue at all — anyone signs in with Google or email, sets up a profile, and they are in."

"They needed to see who is coming to an activity. The admin opens an activity and sees the list of volunteers with their contact details."

"They needed to reach everyone at once. That is announcements, with an optional picture, and volunteers can react so the team can see what is landing."

"And they work across three language communities, so the whole app is in English, isiXhosa and Afrikaans."

The top band wants requirements mapped to features and a justification of how the solution solves the client's problem. The phrase *"no approval queue, because staff time was the bottleneck"* is a justification, not just a feature list. Say that kind of thing.

3. Architecture & system design — Tlhogi

1:30 – 3:15

Targets: Architecture & System Design (15 marks), Database (20), Data Flow (20)

On screen: Your architecture diagram, then the ERD. Then briefly the Android Studio project tree showing `data/model`, `data/repository`, `data/remote`, `data/local`, `data/room`.

"The app is MVVM with a repository layer. Screens never talk to Firebase. A fragment talks to a ViewModel, the ViewModel talks to a repository, and only the repository knows Firestore exists."

"That is why we could swap our image provider without touching a single screen. More on that later."

"The database is Firestore, five collections: users, activities, announcements, activitySignups and the sign-up relationships between them."

"The important design decision is here. A sign-up document is not given a random ID. Its ID is the activity ID and the user ID joined together. That makes it structurally impossible for one volunteer to hold two places on the same activity — not unlikely, impossible, because the second write would target the same document."

"And joining runs in a Firestore transaction. It reads the activity fresh from the server, checks it is still published and still has space, then writes the sign-up and increments the counter together. Either both land or neither does. Two volunteers cannot both take the last spot."

"On DevOps, every push runs a pipeline in GitHub Actions. Kwethu will show it."

The top band is "expert-level technical defense; deep insight into API design, security mitigation and robust DevOps strategy". The deterministic ID and the transaction are the two strongest things you can say in this whole presentation. Slow down for them.

4. Demo 1, the volunteer journey — Kwethu

3:15 – 5:15

Targets: Technical Demo (20 marks), UX & Feedback (10), Visual Design (10)

On screen: The emulator, full screen, already signed in as a volunteer.

Move in this order and do not narrate every tap.

1. **Home.** Greeting, volunteer ID, the four stat tiles, Coming Up, Announcements.
2. **Activities** → **open one** → **Join.** Watch the spot counter drop and the status become Joined.
3. **Schedule.** The activity is now under Upcoming. Tap Completed to show history and hours.
4. **Favourite an activity** from the heart, then the Favourites tab.
5. **Community.** Thumbs up an announcement, and let the count move.
6. **Profile** → **Settings.** Toggle dark mode. Let it redraw on camera.
7. **Settings** → **Language.** Switch to isiXhosa, show one screen, switch back.

"This is the volunteer side. Everything you are seeing is live against Firestore, not mock data."

"When I join, notice the counter drops immediately — that is the transaction Tlhogi described, writing the sign-up and the count together."

"The thumbs up fills straight away, before the server has answered, and the write happens behind it. If it fails the app tells you why rather than silently doing nothing."

"Dark mode and three languages are both real settings, not mock-ups."

Dark mode and the language switch are worth doing on camera because they are visibly *working software*. They feed Visual Design and UX as well as the demo row.

5. Demo 2, admin and the image API — Tlhogi

5:15 – 6:45

Targets: Technical Demo (20 marks), APIs (20), Data Flow (20)

On screen: The emulator signed in as an admin. Have a picture ready in the gallery.

1. **Admin overview.** Volunteer count, published activities, total sign-ups.
2. **Volunteers.** Open one and show their details and programme interests.
3. **Activities** → **open the one Kwethu just joined** → **sign-ups list.** Kwethu's name is there. This is the moment the two halves of the demo connect.
4. **New announcement.** Type a title and body, **attach a picture**, publish.
5. **Back to the volunteer device.** The announcement is there, with the picture.

"Here is the admin side, and this is the same activity Kwethu joined a minute ago. His name is on the sign-up list. Nothing was refreshed by hand."

"Now the picture. This is a third-party REST API, Cloudinary. The app shrinks the photo, sends it as an unsigned upload, and gets back a URL. Firestore only ever stores that URL, never the image itself."

"Unsigned matters. A signed upload needs an API secret, and anything inside an APK can be pulled out by anyone who downloads it. So there is no secret in the app at all — the upload preset on the Cloudinary side controls what is allowed."

"We originally used a different provider and it started refusing our uploads. Because the app talks to an uploader interface rather than to that provider, swapping it was one file and no screen changed."

The APIs row wants "integrates external services" and "excellent error management". The provider-swap story is genuine evidence of good API design, and it actually happened, so you can defend it under questioning.

6. Responsiveness & accessibility — Ash

6:45 – 7:45

Targets: Responsiveness & Accessibility (15 marks)

On screen: The Resizable emulator, on the Activities screen.

1. **Rotate to landscape** (Ctrl+F11). Pause. Rotate back.
2. **Open the Welcome screen and rotate.** This is a genuinely different layout, not a reflow.
3. **Switch Display Mode to Tablet.** The card lists go to two columns.
4. **Settings → Display → Font size**, drag to largest, return to the app.

"The layouts adapt across phone, landscape and tablet."

"This is not thirty-six duplicated layouts. Sizing comes from alternate resource folders, so one set of layouts serves every screen and there is nothing to keep in sync."

"On a tablet the lists become two columns, because one card stretched across a ten-inch screen just looks wrong."

"The Welcome screen has its own landscape layout, because there the button needed to move rather than shrink."

"And accessibility. All text is in scalable units, so at the largest system font size the app still works — you can see it reflow. Every tappable control meets the forty-eight density-pixel minimum from the Android accessibility guidance."

Rotate slower than feels natural. Layout changes happen in a few frames and a marker watching once can miss the point entirely.

7. Security, proven not claimed — Kwethu

7:45 – 9:00

Targets: Security (20 marks), Architecture & System Design (15)

On screen: The Firebase console Rules page, then a terminal running the rules tests, then the passing output.

"Security is enforced by Firestore rules, not by the app. That distinction matters, because an attacker does not have to use our app — they can talk to the database directly."

"A volunteer reads only their own record. An admin reads everyone. A volunteer cannot change their own role, their email, their volunteer ID or their join date. Only you can delete your own account, and an admin cannot delete you."

"On announcements, a volunteer can add and remove their own thumbs up and nothing else. Not somebody else's, not the title, and they cannot pad the count with made-up IDs."

"But we do not want you to take that on trust, so we test it."

[run the tests] "These run our real rules file against the Firestore emulator. Twenty-five tests. Every red permission-denied line you see is the rules correctly refusing an attack."

"No secrets are in the repository either. The signing key and the image service credentials are kept out of version control, and the release build is minified."

Running the tests live is the difference between "strong security practices" and "comprehensive security, advanced mitigation". If you are worried about live demos, record this bit separately and cut it in.

8. GitHub, testing and the pipeline — Kwethu

9:00 – 10:30

Targets: GitHub & Version Control (10 marks), CI/CD (20), Programming Skills (25)

On screen: GitHub → the branch list, then a merged PR, then Actions with a green run expanded to show the four jobs.

"We used Gitflow. Master is the stable branch, develop is what we integrate into, and every feature has its own branch named for the person and the feature."

"Nothing reaches develop without a pull request. Here is one — you can see the commits, the review and the merge."

"On every push and every pull request, this pipeline runs. Four jobs. Unit tests, lint, and the security rules tests all run in parallel, and the APK is only built if all three pass."

"Sixty-six unit tests cover the date and time rules, the models and the error handling. Twenty-five more test the security rules against the emulator. Ninety-one automated tests in total, and a failing test blocks the merge."

"Every run publishes the test report, the lint report and a downloadable APK."

Expand the workflow run so the four job boxes are visible. That picture alone is most of the CI/CD row's top band: "advanced pipeline configuration; comprehensive automated test suite".

9. Hosting and close — all three

10:30 – 11:00

Targets: Technical Demo (20 marks), Communication (10)

On screen: The live hosted download page, then the app installing from it.

Kwethu: *"The app is hosted. This is the live download page, and this is it installing on a real device. Philisa can hand this link to a volunteer today."*

Tlhogi: *"What is next is push notifications for new announcements, and repeating activities so the team does not re-create the same weekly kitchen day."*

Ash: *"Thank you for watching. We are happy to take questions."*

If hosting is not ready by filming day, do not fake it. Show the APK building in the pipeline and say the hosted download is the next step. A marker who clicks a dead link will trust nothing else you showed.

Questions you should be ready for

If they ask	Answer with
Why Firebase and not a REST API with SQL?	A three-person team with no budget and no server to run. Firestore gives authentication, a database and security rules enforced server-side. The security rules <i>are</i> our data integrity constraints, and Cloudinary is a REST API we integrate with.
How do you stop two people taking the last spot?	A Firestore transaction. It reads the activity fresh from the server, and writes the sign-up and the counter together. The rules also refuse a counter change that is not accompanied by the matching sign-up.

What stops a volunteer making themselves an admin?	The rules refuse any update that changes the role field. There is a test for exactly that, and it runs on every push.
Is the API key in the app safe?	There is no secret in the app. The Cloudinary preset is unsigned, so the limits live on their side. Our signing key and credentials are gitignored.
What was the hardest bug?	An end time entered as 04:00 instead of 16:00 made an activity look finished before it started, and hours came out as zero. Now the app refuses an end time that is not after the start, and falls back safely on old data. There are unit tests for both.
How did you split the work?	Be honest and specific. Name the branches.

Delivery notes

- **Hand over by name.** “Tlhogi will take you through the architecture.” It reads as a team rather than three separate videos.
- **Do not read this script on camera.** Markers can hear it. Learn the shape of each section.
- **Say the number.** “Ninety-one automated tests” lands harder than “we wrote tests”.
- **Silence while something loads is fine.** Filling it with “um, so this is just loading” is what costs marks.
- **Never apologise for the app on camera.** If something breaks, carry on and mention it afterwards if it matters.
- **Watch it back once** before submitting, with the rubric next to you.